

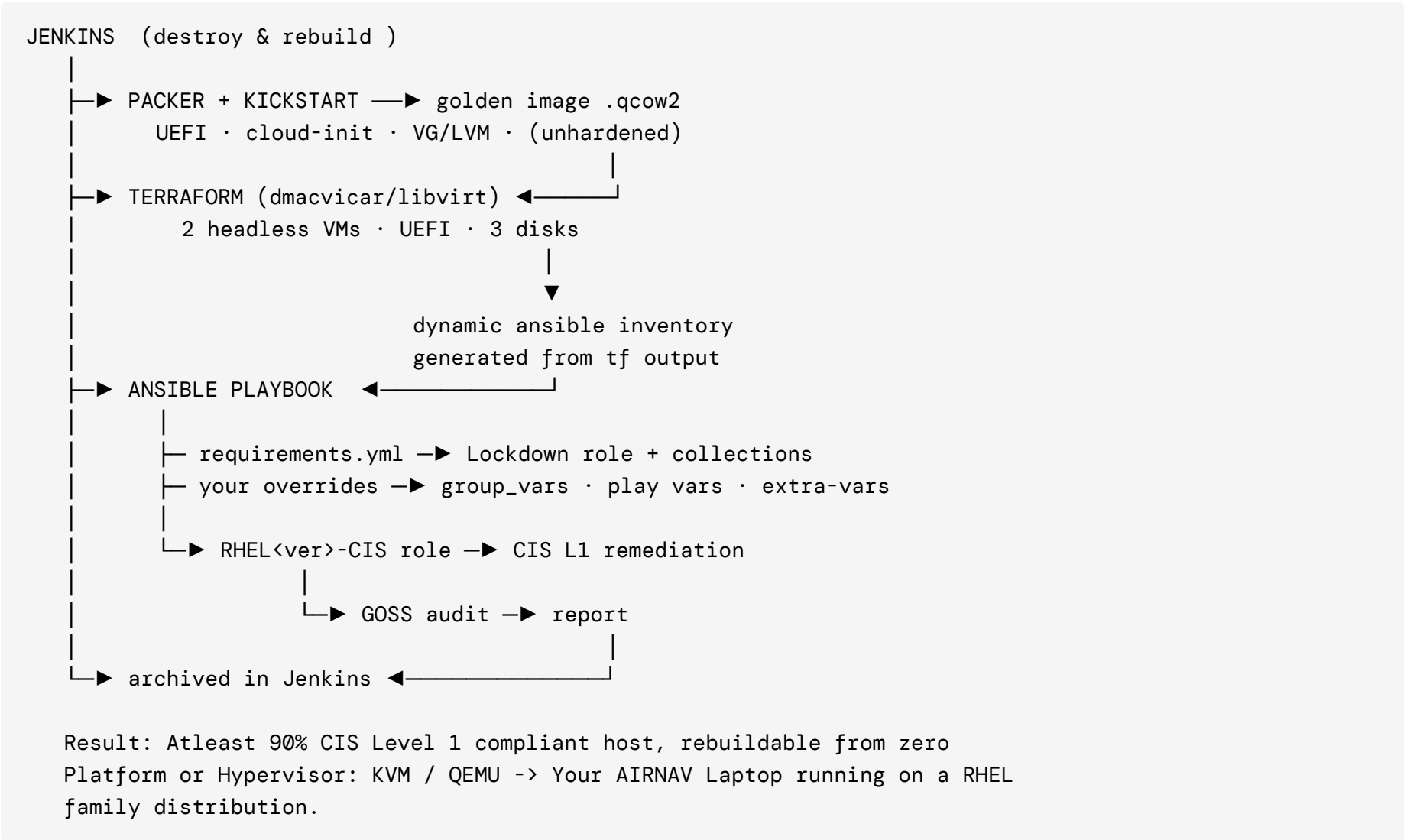
Trainee Concepts Guide

Trainee Module: Golden Image to Hardened VM Pipeline

Duration: 2 to 3 weeks
Format: Individual build, individual presentation
Companion doc: *Concepts You Need to Learn* (read that first, it maps every concept this module touches)

1. What you are building

By the end of this module you will have a working pipeline that takes an AlmaLinux ISO and ends with a **headless, hardened, compliance-scanned VMs, all triggered from a single Jenkins build.**



The mental model: each tool does one job and hands off an artifact to the next.

- Packer produces an **image**.
- Terraform produces **infrastructure**. It answers "how many machines, with what hardware?"
- Ansible produces **configuration**. It answers "what state should this running machine be in?"
- Goss produces **evidence**. It answers "can we prove it?"

Part 1: Virtualization and KVM fundamentals

1.1 The stack

What it is: KVM is the kernel module that makes the CPU do virtualization. QEMU emulates the rest of the machine. libvirt is the management layer that both Cockpit and Terraform talk to.

What to learn

- ☐ How KVM, QEMU, and libvirt relate to each other
- ☐ What a **domain** is in libvirt terms (spoiler: it's a VM)
- ☐ Domain XML: where the config for a VM actually lives
- ☐ What `virsh` is and that Cockpit is just a web UI on top of the same libvirt API
- ☐ Why we use Cockpit and virsh instead of virt-manager on servers

1.2 virsh commands you'll actually use

- ☐ `virsh list --all`, `virsh start/shutdown/destroy <domain>`
- ☐ `virsh dumpxml <domain>` and reading the output
- ☐ `virsh domblklist <domain>` to see disks
- ☐ `virsh attach-disk` and `virsh detach-disk` on a running VM
- ☐ `virsh pool-list --all`, `virsh vol-list <pool>`, `virsh pool-dumpxml <pool>`
- ☐ `virsh net-list --all`, `virsh net-dhcp-leases <net>` (this one is how you find VM IPs)

1.3 Storage pools and volumes

What it is: a pool is where libvirt keeps disk images. A volume is one disk image inside it.

What to learn

- ☐ The difference between a pool and a volume
- ☐ Directory-backed vs LVM-backed pools, at a high level
- ☐ What qcow2 is, and how it differs from raw
- ☐ Thin provisioning: why a 50G qcow2 file can be 2G on disk

1.4 BIOS vs UEFI

What to learn

- ☐ What UEFI is and how it differs from legacy BIOS boot
- ☐ What an **EFI System Partition** (ESP) is, why it's FAT formatted, why it mounts at `/boot/efi`
- ☐ What OVMF is: `OVMF_CODE.fd` (the firmware, shared and read-only) and `OVMF_VARS.fd` (per-VM NVRAM, writable)
- ☐ Why each VM needs its own NVRAM file
- ☐ What Secure Boot is and why we're leaving it out of scope for now

Part 2: Kickstart and golden images

2.1 Unattended installation

What it is: Kickstart is Red Hat's answer file format. The installer reads it and installs without asking you anything.

What to learn

- ☐ How the installer finds the kickstart file (`inst.ks=` on the kernel command line)
- ☐ Kickstart sections: `commands`, `%packages`, `%pre`, `%post`, `%end`
- ☐ Partitioning directives: `part`, `volgroup`, `logvol`, `--ondisk`, `--grow`, `--size`
- ☐ Creating an ESP: `part /boot/efi --fstype=efi`
- ☐ `%post` scripts and when they run (chroot vs not)
- ☐ `ksvalidator` to check your file before wasting a build

2.2 Partitioning, LVM, filesystems

What it is: LVM sits between your physical disks and your filesystems so you can resize things later without repartitioning.

What to learn

- ☐ PV, VG, LV: what each layer is and how they stack
- ☐ Why we use LVM instead of plain partitions
- ☐ `pvs`, `vgs`, `lvs`, `lvextend`, `xfs_growfs`, `resize2fs`
- ☐ xfs vs ext4 at a practical level
- ☐ Mount options that matter for hardening: `nodev`, `nosuid`, `noexec`
- ☐ **Why CIS wants separate partitions** for `/var`, `/var/log`, `/var/log/audit`, `/tmp`, `/home`. Hint: think about what happens when one of them fills up.
- ☐ **Thick vs Thin Logical Volumes**
- ☐ File system layering - **Host image file, Guest VG, Guest filesystem**
- ☐ `/tmp` as a **tmpfs** via the systemd `tmp.mount` unit instead of a logical volume, and the tradeoff (it lives in RAM and is wiped on reboot)

2.3 Golden images

What it is: a pre-built, known-good OS image you clone instead of installing from scratch every time.

What to learn

- ☐ Why golden images exist: speed, consistency, reproducibility
- ☐ What belongs in an image (base packages, agents, users) vs what belongs in configuration management (hardening, app config, anything environment specific)
- ☐ Image versioning: why the artifact should be tagged and traceable
- ☐ What **cloud-init** does on first boot and why it makes a single image reusable
- ☐ Why you generally don't bake SSH host keys or machine IDs into an image

Can you answer this?

- Why not bake the CIS hardening straight into the golden image? Give one argument for baking it in and one for applying it after.

Part 3: Hashicorp Packer

What it is: Packer automates building the golden image. It boots an ISO in a VM, feeds it the kickstart, waits, and saves the resulting disk.

What to learn

- ☐ HCL2 syntax: `source`, `build`, `variable`, `locals`
- ☐ The QEMU builder and its key options
- ☐ `http_directory`: how Packer serves your kickstart file to the installer over HTTP
- ☐ `boot_command`: how the kickstart URL gets typed onto the boot line, and why timing and `boot_wait` matter
- ☐ UEFI settings: `efi_boot`, `efi_firmware_code`, `efi_firmware_vars`
- ☐ Provisioners (shell, ansible) and when to use them instead of `%post`
- ☐ `packer init`, `packer validate`, `packer fmt`, `packer build`
- ☐ Where the output artifact lands and how to control that

Part 4: Hashicorp Terraform

4.1 Core concepts

What it is: Terraform declares what infrastructure should exist, then makes reality match.

What to learn

- ☐ Declarative vs imperative: you describe the end state, not the steps
- ☐ **State:** what `terraform.tfstate` is, why it exists, why losing it is bad, and why it must not live somewhere that gets wiped
- ☐ The workflow: `init`, `validate`, `plan`, `apply`, `destroy`
- ☐ Providers and how `required_providers` pins them
- ☐ Resources vs data sources
- ☐ Variables, outputs, locals
- ☐ `count` and `for_each` for making more than one of something
- ☐ Implicit dependencies (referencing another resource) vs `depends_on`
- ☐ Functions you'll use: `templatefile()`, `file()`, `join()`, splat expressions like `resource[*].attribute`

4.2 The libvirt provider

- ☐ `libvirt_pool`, `libvirt_volume`, `libvirt_cloudinit_disk`, `libvirt_domain`
- ☐ Base volume vs cloned volume: how you make N VMs from 1 image
- ☐ Adding extra disks to a domain and setting the bus (virtio vs virtio-scsi)
- ☐ UEFI on a domain: the `firmware` and `nvrwam` blocks
- ☐ Getting VM IPs out of the provider (and why they may not be available immediately)
- ☐ What a clean destroy should leave behind (nothing)

4.3 Generating the inventory

- ☐ Using `templatefile()` plus `local_file` to write an ansible inventory from Terraform outputs
- ☐ Static vs dynamic inventory: the tradeoff

Part 5: Security Hardening and CIS

Read this before the Ansible section. The Lockdown role is just an implementation of what's below.

What it is: the Center for Internet Security publishes **Benchmarks**, prescriptive config guides per platform. Download the RHEL 9 or RHEL 10 PDF for your assigned OS and skim it. You don't need to read all 800 pages, but you do need to read a few recommendations end to end and understand the shape of one.

What to learn

- Why hardening exists: attack surface, defense in depth, least privilege
- Anatomy of one recommendation: title, rationale, **impact**, audit procedure, remediation. The `impact` field is where the benchmark tells you what will break, and it's the one people skip.
- **Level 1 vs Level 2:** L1 is the practical baseline, L2 is defense in depth with real operational cost
- **Server vs Workstation** profiles. We use Level 1, Server.
- **Benchmark versioning:** rules get renumbered and reclassified between releases, so a rule ID means nothing without a version. This is why the role gets pinned.
- The rule categories you'll meet: filesystem and mount options, package and service minimization, systemctl, firewalld, SELinux, PAM and password policy, sshd, auditd and logging, file permissions, time sync, banners
- The four that lock you out: **sshd, PAM, firewall, mount options**
- **Tailored baseline:** switching rules off is legitimate when documented. An undocumented failure and a deliberate exception look identical in a report and are completely different in an audit.
- **Compliance is not security.** 100% means you match one baseline at one moment. 90% with documented exceptions can be the stronger posture.
- Names you'll hear: **DISA STIG** (DoD equivalent, stricter), **SCAP / OpenSCAP** (machine-readable scanning), **NIST 800-53 / ISO 27001** (control frameworks, the *what* not the *how*)

Read

- CIS Benchmarks (free PDF after signup): <https://www.cisecurity.org/cis-benchmarks>
- CIS Controls: <https://www.cisecurity.org/controls>
- Your role's README and `defaults/main.yml`: <https://github.com/ansible-lockdown>
- OpenSCAP: <https://www.open-scap.org/>

Part 6: Ansible

This is the biggest section and the most important one. If you only deeply learn one tool this module, make it this one.

6.1 Fundamentals

- Agentless architecture: it's just SSH and Python on the target
- **Idempotency:** what it means and why `changed` vs `ok` matters
- Modules vs tasks vs plays vs playbooks
- `ansible.cfg` and what it controls
- Ad-hoc commands: `ansible all -m ping`

6.2 Inventory

- INI vs YAML inventory format - YAML would be much more preferred
- Hosts, groups, nested groups, `children`
- Special groups: `all`, `ungrouped`
- Host variables vs group variables
- `group_vars/` and `host_vars/` directory layout, and how files there are picked up automatically
- `group_vars/all/` as a directory instead of a single file, and why that's handy when one file is vaulted
- Dynamic inventory: what it is and when you'd want it

6.3 Play and playbook structure

- ☐ Anatomy of a play: `hosts`, `become`, `vars`, `roles`, `tasks`, `handlers`
- ☐ Task structure: `name`, `module`, `arguments`, `when`, `loop`, `register`, `tags`, `notify`
- ☐ Handlers: what makes them different from tasks and when they run
- ☐ `pre_tasks`, `roles`, `tasks`, `post_tasks` execution order
- ☐ Tags and `--tags` / `--skip-tags`, which the Lockdown roles rely on heavily
- ☐ `--check` and `--diff` for dry runs
- ☐ Facts: what `gather_facts` collects and how to use `ansible_facts` in conditions

6.4 Roles and requirements.yml

- ☐ Role directory structure: `defaults/`, `vars/`, `tasks/`, `handlers/`, `templates/`, `files/`, `meta/`
- ☐ **The difference between `defaults/main.yml` and `vars/main.yml`**
- ☐ `requirements.yml`: declaring external roles and collections
- ☐ `ansible-galaxy install -r requirements.yml -p roles/`
- ☐ Pinning a role version, and why "main" is risky in production
- ☐ Why you consume an upstream role rather than forking and editing it

6.5 Variable precedence

This is the concept the module is built around. Learn it properly.

- ☐ The full precedence order, from role defaults at the bottom to `--extra-vars` at the top
- ☐ Where these sit relative to each other: role defaults, inventory group vars, `group_vars/all`, `group_vars/<group>`, `host_vars`, play vars, `vars_files`, task vars, `include_vars`, role vars/, `set_fact`, `--extra-vars`
- ☐ Why `--extra-vars` always wins
- ☐ Why role `vars/main.yml` is hard to override and `defaults/main.yml` is easy
- ☐ Variable scoping: global, play, host
- ☐ How the same variable set in four places resolves to one value

Exercise before you touch the Lockdown role: create a throwaway role with one variable in `defaults/main.yml`. Set it also in `group_vars/all`, in the play's `vars:`, and via `--extra-vars`. Print it with `debug` and remove one layer at a time.

Can you answer this?

- Same variable in role defaults, `group_vars/all`, play `vars:`, and `--extra-vars`. Which wins? Now remove extra-vars. Now remove the play vars.
- Why would a role developer put a variable in `vars/` instead of `defaults/`?

6.6 Templates and Jinja2

- ☐ `template` vs `copy` modules
- ☐ Jinja2 basics: `{{ }}`, `{% %}`, `for`, `if`
- ☐ Filters you'll meet: `default()`, `join()`, `bool`, `to_nice_yaml`
- ☐ When a `lineinfile` be used in comparison to Jinja templates

6.7 ansible-vault

- ☐ What vault encrypts and what it doesn't
- ☐ `ansible-vault create` / `edit` / `encrypt` / `decrypt` / `view` / `rekey`
- ☐ Encrypting a whole file vs a single variable (`encrypt_string`)
- ☐ `--vault-password-file` and `--ask-vault-pass`
- ☐ Vault IDs, at least conceptually
- ☐ Keeping the vault password out of Git and out of Jenkins logs (Jenkins credentials, not a hardcoded string)
- ☐ The convention of `group_vars/all/vars.yml` pointing at `group_vars/all/vault.yml`

6.8 Ansible Lockdown specifically

- ☐ What CIS Benchmarks are and who publishes them
- ☐ **Level 1 vs Level 2**: L1 is the baseline that shouldn't break normal use, L2 is defense in depth with real operational cost
- ☐ Server vs Workstation profiles
- ☐ How the role structures rules by section number and tags them
- ☐ Reading `defaults/main.yml` in a Lockdown role
- ☐ The `*_rule_x_y_z` per-rule toggles
- ☐ Which rules are known to be disruptive (SSH, PAM, firewall, mount options). These are the ones that lock you out.
- ☐ `setup_audit` and `run_audit` for the built-in Goss audit
- ☐ The concept of a **tailored baseline**: not every rule fits every environment, and documenting an exception is a legitimate answer

Can you answer this?

- Your hardening run breaks SSH access halfway through. Which rule categories are the likely culprits, and how do you get back in without recreating the VM?
 - What's the difference between a rule failing because it's broken and a rule failing because you deliberately tailored it off?
-

Part 7: Goss and compliance

What it is: Goss is a lightweight tool that checks the state of a system against a YAML spec. The Lockdown roles ship with Goss specs matching the CIS rules.

What to learn

- ☐ The difference between **remediation** (Ansible changing things) and **audit** (Goss checking things)
- ☐ Why you audit with something other than the tool that did the remediation
- ☐ Goss spec structure: `file`, `package`, `service`, `command`, `user`, `port` checks
- ☐ Reading the audit output: pass, fail, skip
- ☐ Output formats and producing a machine-readable report
- ☐ Where the report lands and how to get it back to the controller
- ☐ Why 100% is rarely the goal, and what a documented exception looks like

Can you answer this?

- Take one failing check. Is the right answer to remediate, to tailor a variable, or to document an exception? Who would sign off on that exception in a real organization?
-

Part 8: Jenkins and pipeline concepts

What it is: Jenkins runs your pipeline. The pipeline is code, checked into your repo as a `Jenkinsfile`.

What to learn

- ☐ Declarative pipeline syntax: `pipeline`, `agent`, `stages`, `stage`, `steps`, `post`
 - ☐ `parameters {}`: `booleanParam`, `string`, `choice`
 - ☐ `when {}` conditions to skip stages based on parameters
 - ☐ `environment {}` and injecting variables into tool calls
 - ☐ **Jenkins credentials**: storing your vault password and SSH keys properly
 - ☐ `archiveArtifacts` for the Goss report
 - ☐ Workspace lifecycle and where persistent things like Terraform state should live instead
 - ☐ `post { always { } }` for cleanup that runs even on failure
 - ☐ Why the pipeline should fail loudly rather than continue quietly
-

Part 9: Cross-cutting and stretch

9.1 Git

- ☐ Branching, meaningful commit messages, `.gitignore`
- ☐ What should never be committed: keys, vault passwords, `.tfstate`, build artifacts
- ☐ Why commit history is a form of documentation

9.2 Linting and quality

- ☐ `ansible-lint`, `yamllint`
- ☐ `terraform fmt` and `terraform validate`
- ☐ `packer fmt` and `packer validate`
- ☐ `ksvalidator`
- ☐ `pre-commit-config.yaml`

9.3 Stretch goals if you finish early

- ☐ Add Level 2 hardening to a second VM and compare the breakage
- ☐ Add a pre-hardening Goss run so you can show a before and after score
- ☐ Generate an SBOM (Software Bill of Materials) out of your golden image
- ☐ Parameterize your Jenkins pipeline to pick the OS version at build time and **hard fail the pipeline if it's on the wrong distribution, version, family *FAIL HARD, FAIL FAST*** logic

Reference links

Ansible

- Docs home: <https://docs.ansible.com/ansible/latest/>
- Variable precedence: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_variables.html#variable-precedence-where-should-i-put-a-variable
- Inventory: https://docs.ansible.com/ansible/latest/inventory_guide/intro_inventory.html
- Roles: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_reuse_roles.html
- Vault: https://docs.ansible.com/ansible/latest/vault_guide/index.html
- ansible-lint: <https://ansible.readthedocs.io/projects/lint/>

Ansible Lockdown & CIS Benchmarks

- Org: <https://github.com/ansible-lockdown>
- RHEL9-CIS: <https://github.com/ansible-lockdown/RHEL9-CIS>
- RHEL10-CIS: <https://github.com/ansible-lockdown/RHEL10-CIS>
- CIS Benchmarks: <https://www.cisecurity.org/cis-benchmarks>

Packer

- Docs: <https://developer.hashicorp.com/packer/docs>
- QEMU builder: <https://developer.hashicorp.com/packer/integrations/hashicorp/qemu>

Terraform

- Docs: <https://developer.hashicorp.com/terraform/docs>
- libvirt provider: <https://registry.terraform.io/providers/dmacvicar/libvirt/latest/docs>

Kickstart and LVM

- Kickstart reference: <https://pykickstart.readthedocs.io/en/latest/kickstart-docs.html>
- AlmaLinux docs: <https://wiki.almalinux.org/>

Virtualization

- libvirt: <https://libvirt.org/docs.html>
- virsh manpage: <https://libvirt.org/manpages/virsh.html>
- Cockpit: <https://cockpit-project.org/documentation.html>

Goss and Jenkins

- Goss: <https://github.com/goss-org/goss>
- Jenkins pipeline syntax: <https://www.jenkins.io/doc/book/pipeline/syntax/>
- Jenkins credentials: <https://www.jenkins.io/doc/book/using/using-credentials/>